



LAB REPORT

Course Code : ICE 3146 **Course Title :** Microprocessor and Interfacing Laboratory

Lab Task No. : 07

Lab Task Topic: Interfacing and Operating a Dot Matrix Display Using the 8086 Microprocessor Kit.

No.	Evaluation Criteria with Marks	Put Tick (✓) Mark					Marks Obtained	Remarks
		Excellent	Good	Fair	Poor	Fail		
REPORT WRITING								
1	Theory & Objectives (2.5)							
2	Materials & Methods (5)							
3	Data Analysis & Discussion/Diagram (10)							
4	Precautions and Conclusion (2.5)							
5	Time Management (5)							
	TOTAL							
LAB PERFORMANCE								
1	Preparation for Lab (5)							
2	Experiment and Engagement (15)							
3	Communication (5)							
	TOTAL							
24 th June, 2026 Date of Experiment		11 th July, 2026 Date of Submission					_____ Teacher’s Signature	

Semester : Summer **Year :** 2026 **Level-Term :** L2-T2 **Section :** A2

Submitted to : **Sameer Khairul**
Lecturer, Dept. of Information and Communication Engineering
Daffodil International University

Submitted by : **Tanzim Ahamed**
Reg.No. : 242-50-036
Dept. of Information and Communication Engineering
Daffodil International University

Title: Interfacing and Operating a Dot Matrix Display Using the 8086 Microprocessor Kit.

Abstract:

This experiment was performed using the Intel 8086 Microprocessor Trainer Kit to interface and operate an 8×8 dot matrix display by using assembly language instructions. The main objective of this experiment was to understand how the 8086 microprocessor communicates with a dot matrix display through the 8255 Programmable Peripheral Interface (PPI), how row and column data are controlled, and how a character pattern is displayed by rapid scanning. In this experiment, the 8255 PPI was initialized first, the output ports were configured, and a predefined font pattern stored in memory was sent repeatedly to the display. As a result, the dot matrix module produced the pattern of the capital letter A. The experiment helped to understand port addressing, dot matrix multiplexing, memory-based pattern generation, delay control, and practical hardware interfacing using the 8086 microprocessor.

Introduction (Theory):

The Intel 8086 is a 16-bit microprocessor that can communicate with external hardware devices through Input/Output (I/O) ports. Devices such as LEDs, seven-segment displays, LCDs, and dot matrix modules are common output peripherals. Connecting these devices with the microprocessor is called interfacing. In assembly language, the OUT instruction is used to transfer data from a register to an output port and thereby control external hardware.

An 8×8 dot matrix display consists of 64 LEDs arranged in 8 rows and 8 columns. By turning ON selected LEDs at the intersections of rows and columns, different symbols, characters, and simple graphics can be displayed. The LEDs are not usually driven all at once. Instead, the display is operated by a scanning or multiplexing technique in which one row or one column is activated at a time while the corresponding pattern data is sent to the other lines. Because the scanning happens very quickly, the human eye sees a steady complete pattern due to persistence of vision.

In this experiment, the 8255 PPI was used as the interfacing device. Port B was used to send the font data, Port C was used for row selection, and Port A was set appropriately for the connected display hardware. A font table containing eight binary values was stored in memory. These values represented the required pattern for the displayed character. The program selected one row at a time by rotating a bit through register AH and simultaneously sent the corresponding font byte to the dot matrix through the output ports. The rapid repetition of this process formed the capital letter A on the display.

Material:

Hardware	Software
1. Intel 8086 Microprocessor Trainer Kit 2. 8×8 Dot Matrix Display Module 3. Interface Connections / Trainer Board 4. Power Supply	1. Assembly Program 2. On-board Monitor / Kit Commands 3. MASM and MDA-8086 (if PC-assisted entry is used)

Method:

Code:

```
CODE SEGMENT
    ASSUME CS:CODE,DS:CODE,ES:CODE,SS:CODE
PPIC_C    EQU    1EH
PPIC       EQU    1CH
PPIB       EQU    1AH
PPIA       EQU    18H
;
; ORG    1000H
MOV    AL,10000000B
OUT    PPIC_C,AL
;
MOV    AL,11111111B
OUT    PPIA,AL
;
L1:       MOV    SI,OFFSET FONT
;
MOV    AH,00000001B
;
L2:       MOV    AL,BYTE PTR CS:[SI]
OUT    PPIB,AL
;
MOV    AL,AH
OUT    PPIC,AL
CALL    TIMER
INC    SI
CLC
ROL    AH,1
JNC    L2
JMP    L1
;
INT    3
;
TIMER:    MOV    CX,300
TIMER1:   NOP
NOP
NOP
NOP
LOOP    TIMER1
RET
;
FONT:     DB    11111111B
          DB    11000000B
          DB    10110111B
          DB    01110111B
          DB    01110111B
          DB    10110111B
          DB    11000000B
          DB    11111111B
CODE      ENDS
END
```

Execution (PC Mode):

1. First, the assembly language program was written in Notepad and saved as .asm file.
2. Machine code execution using CMD-

```
C:\MDA\8086\Asm8086>MASM.exe SUM26A17.asm
```

```
Object filename [SUM26_1.OBJ]: SUM26A17
```

```
Source listing [NUL.LST]: SUM26A17
```

```
Cross-reference [NUL.CRF]:
```

```
0 Warning Errors
```

```
0 Severe Errors
```

1. The MDA (Microprocessor Development and Analysis) software was opened.
2. The SUM26A13.asm file was loaded using the CMD panel (LO).
3. Then in MDA terminal- LOD186 SUM26A17.ASM executed.
4. Then in MDA 8086 from FILE DOWNLOAD the SUM26A17.ABS file opened and Run executed.
5. The output and register values were observed through the simulator window and the DOT Matrix Display shows the output according the code.

Result:

Result for PC Mode:

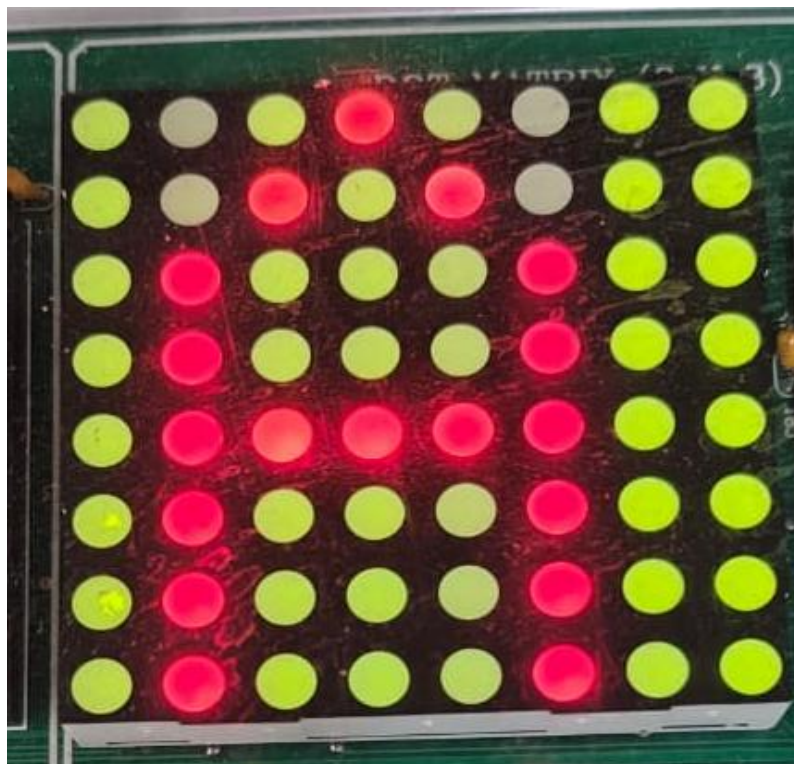


Fig 7.1: DOT Matrix Display outputs for different Sequence of Code.

Observed ROW/Pattern Data:

Step	Row Select Mask	FONT Byte Sent to Port B
1	00000001B (01H)	11111111B (FFH)
2	00000010B (02H)	11000000B (C0H)
3	00000100B (04H)	10110111B (B7H)
4	00001000B (08H)	01110111B (77H)
5	00010000B (10H)	01110111B (77H)
6	00100000B (20H)	10110111B (B7H)
7	01000000B (40H)	11000000B (C0H)
8	10000000B (80H)	11111111B (FFH)

Discussion:

In this experiment, an 8×8 dot-matrix display was interfaced with the 8086 microprocessor through the 8255 Programmable Peripheral Interface. The control word 80H configured the required 8255 ports as output ports, while FFH was sent to Port A during initialization. The column-pattern data stored in the FONT table were sent sequentially to Port B.

The row-selection process started with the mask 00000001B. After displaying each row, the ROL AH,1 instruction shifted the active bit to select the next row. Thus, all eight rows were scanned sequentially using masks from 01H to 80H. A delay subroutine controlled the scanning speed, and the complete sequence was repeated continuously.

Because the rows were refreshed rapidly, the individual row outputs appeared as one stable pattern due to persistence of vision. The observed dot-matrix output matched the programmed FONT data, confirming that the port configuration, row scanning, column encoding, rotation instruction, and delay routine operated correctly. This experiment demonstrated practical dot-matrix multiplexing and peripheral control using 8086 assembly language.